

WHAT THIS SESSION IS

The first half wraps up the remaining theory (a few NLP terms and the full transformer pipeline). The second half is the first real coding: setting up API keys and writing a plain Java program that calls OpenAI and Gemini, first by hand and then with an SDK. If you are comfortable with Session 01, you can skim the recap and focus from Section 4 onward.

CONTENTS

1. Quick recap of Session 01
2. More NLP tasks: POS, NER, sentiment, classification
3. The full transformer pipeline, step by step
4. What an LLM really is
5. Closed source versus open source, and running models locally
6. Getting your API keys (OpenAI and Gemini)
7. Hands on: calling OpenAI from plain Java (no SDK)
8. System prompt versus user prompt
9. Reusing the same code for Gemini
10. Hands on: the official OpenAI Java SDK
11. Practical tips and gotchas

1 Quick recap of Session 01

A few points were reinforced at the start:

- **AI will not replace traditional software.** Even ChatGPT is a mix of both. The AI model produces the answer, but the login (OAuth), the UI, and the payment flow are all

ordinary rule based software written by developers. Modern products combine the two.

- **Every modern model uses transformers** behind the scenes.
- **Tokenization is model specific.** There is no single rule that a space always means a new token. Different companies use different tokenizers, so "I love learning AI models" can split differently in each one. When asked, ChatGPT itself confirmed that different models do not always use the same tokenizer.
- **More dimensions in a vector means more information** stored about a word, which helps prediction. Note the instructor's careful wording: we say "better prediction", not "accuracy", because even today models still hallucinate.

2 More NLP tasks: POS, NER, sentiment, classification

These are classic tasks that live under the NLP family. They existed before transformers (as "traditional NLP") and are now handled inside the transformer architecture.

Example sentence: *"Apple announced a new iPhone in California on Friday."*

Task	What it does	On the example
POS tagging (Part of Speech)	Assigns a grammatical role to each word	"Apple" is a noun, "announced" is a verb, "iPhone" is a noun
NER (Named Entity Recognition)	Identifies real world entities	"Apple" is an organisation, "iPhone" is a product, "California" is a location, "Friday" is a date
Sentiment analysis	Detects the emotional tone (positive or negative)	"The iPhone is absolutely amazing" is positive; "the battery life is terrible" is negative
Text classification	Assigns a predefined category to the whole text	"Virat Kohli scored a century" is sports news; "you won \$10,000, click here" is spam

NOTE THE DIFFERENCE

NER labels individual entities inside a sentence. Text classification labels the entire message with one category. In "Virat Kohli scored a century", NER sees that "Virat Kohli" is a person, while classification decides the whole sentence is sports news.

3 The full transformer pipeline, step by step

Putting Session 01 together, here is what happens when you send a prompt such as *"Who is Virat Kohli and which sport does he play and for which country?"*

1. Text input your natural language prompt
2. Tokenization split the text into tokens
3. Token IDs map each token to an integer ID
4. Embeddings convert IDs into dense vectors (thousands of dimensions)
5. Positional encoding inject word order (dog=1, bites=2, man=3)
6. Self-attention each token looks at the others to fix meaning
7. Multi-head attention many attention heads look in parallel
8. Encoder (BERT-like) understands the full context
9. Decoder (GPT-like) generates the output word by word
10. Output the predicted response you read

Two new pieces compared to Session 01:

- **Positional encoding** adds the position of each word before attention runs, so the model knows word order. "Dog bites man" and "man bites dog" are very different, and position is how it tells them apart.
- **Attention scores.** In "Who is Virat Kohli...", the token "Virat Kohli" gets a high attention score because it is the important part, while filler tokens like "is" and "which" get lower scores. The word "he" is correctly linked back to Virat Kohli. Different heads can focus on the person, the sport, the country, and the relationships all at once.

4 What an LLM really is

DEFINITION

An **LLM (Large Language Model)** is a transformer based neural network trained on a massive amount of data (billions of documents). "Large" refers to the huge amount of training data. In the simplest personal framing from the class: an LLM is just a **large scale predictor**. It predicts the next token, again and again.

Important: an LLM does **not** store your data in a database and look it up like traditional software. If it did, we would need endless servers and hand written rules. It simply learned patterns from training data and now predicts. (Later in the course you will see vector stores and vector databases, which are a different thing used to give models extra knowledge.)

5 Closed source versus open source, and running models locally

Closed source (frontier) models	Open source models
Served over an API. Need an API key. Priced per token.	Free to download and run locally. Full control, including fine tuning.
Examples: OpenAI GPT, Google Gemini, Anthropic Claude.	Examples: Mistral, Gemma, Qwen, GLM, Llama.

A useful side note from the class: try not to stick to just one model. Different models tend to suit different tasks.

Tools to run models on your own machine

- **Ollama:** an open source tool. Install it, then `ollama run <model>` to pull and chat with a model. Recommended starting point.
- **LM Studio:** a very good tool, but the tool itself is not open source. Fine for developers who do not mind that.

PICKING A LOCAL MODEL FOR YOUR HARDWARE

Smaller models such as **Mistral** run on almost any GPU. **Gemma** (for example the roughly 9 to 10 GB build) needs about 12 GB of GPU memory. **Qwen** has a coder tuned variant that is good for coding but heavy (around 27 to 29 GB). Very large open models like GLM can be hundreds of GB and only run on cloud or multi GPU servers. On Apple silicon, look for the **MLX** builds, which are optimised for macOS. The word **quantization** (shrinking a model to run on smaller hardware) will be covered in the fine tuning classes.

HONEST ADVICE FROM THE SESSION

While you are learning, do not chase free options. Add a small amount (around \$5) to an API account and use a frontier model. Free keys sometimes work and sometimes fail for no clear reason, and as a beginner you cannot tell if the problem is you or the model. Once you are an expert, then experiment with local and free setups. Local models mainly matter for privacy and for fine tuning, both of which come later.

6 Getting your API keys

OpenAI (paid, per token)

1. Search for **OpenAI** and open the **API platform** (not ChatGPT, not Codex). Log in with the API platform option.
2. Open the account menu (bottom left), go to **organization settings**, then **Billing**. Add a small balance (about \$5 is plenty for the whole course). Pricing is shown per one million tokens for each model, covering both input and output tokens.
3. Go to **API keys** and click **Create new secret key**. Copy it immediately and store it somewhere safe. Once you close the dialog in OpenAI, you cannot view the secret again (same behaviour as AWS secret keys).

TWO SAFETY RULES

Turn **auto recharge off** in billing, so a leaked key cannot silently keep charging you. And never hard code your key in source that goes to GitHub.

Pricing reference: <https://openai.com/api/pricing/>

Gemini (free key)

1. Search for **Google AI Studio** and open it. You are usually already logged in with your Google account.
2. Click **Create API key**. That is it, no billing setup needed. It is free, though a free key is not guaranteed to always respond.

Google AI Studio: <https://aistudio.google.com/>

7 Hands on: calling OpenAI from plain Java (no SDK)

The goal of the first demo is deliberately low level: a plain Maven Java project (not Spring Boot) that uses only the built in `java.net.http` classes to call the OpenAI API. No third party library at all. The mental model is simple: your program needs a "browser" (an HTTP client), it makes a "request", and it reads a "response".

Step 1: read the key from an environment variable

Do not hard code the key. In IntelliJ, open **Edit Configurations**, add an **environment variable** named `OPENAI_API_KEY`, and paste your key as the value. Then read it in code:

```
String apiKey = System.getenv("OPENAI_API_KEY");
```

Step 2: create the client (the browser)

```
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.URI;

HttpClient client = HttpClient.newHttpClient();
```

Note the imports come from core Java (`java.net.http`), not from Spring, Jakarta, or any external library.

Step 3: build the request body (JSON)

The API speaks JSON. The body specifies the model and a list of messages. Each message has a **role**. Using a Java text block keeps it readable:

```
String requestBody = """
    {
      "model": "gpt-4o",
      "messages": [
        { "role": "system", "content": "You are a top movie review expert." },
        { "role": "user", "content": "Name a movie that a software engineer
must watch." }
      ]
    }
    """;
```

Step 4: build and send the request

This is a **POST** request because we are sending data (the prompt) in the request body. Two headers are required: the content type and the authorization header carrying your key as a bearer token.

```
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.openai.com/v1/chat/completions"))
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(requestBody))
    .build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

System.out.println(response.body());
```

The response comes back as JSON. It includes the generated text plus usage information (for example how many tokens were consumed). You can parse it with the Java Stream API to pull out just the message content if you do not want the whole payload.

8 System prompt versus user prompt

THE TWO ROLES THAT MATTER NOW

system is set by you, the developer, to define how the AI should behave ("You are a top movie review expert", "Act as a senior software engineer"). **user** is what the end user actually asks. Giving a clear system role is a good prompt engineering habit that leads to more precise, focused answers.

9 Reusing the same code for Gemini

Because this raw approach just makes an HTTP call, the very same client and request code works for Gemini. You only change three things: the API key, the URL, and the model name. Google exposes an **OpenAI compatible endpoint** so the request shape stays identical:

```
// URL:
https://generativelanguage.googleapis.com/v1beta/openai/chat/completions
// model: "gemini-2.5-flash" (a fast, low cost model)
// key:   your free Gemini key from Google AI Studio
```

WHY "OPENAI COMPATIBILITY"?

It does not mean Google needs OpenAI's technology. The OpenAI request format simply became an industry standard, so Google offers a compatible layer to make adoption easy. Gemini also works without it, using its own native format.

10 Hands on: the official OpenAI Java SDK

Writing raw HTTP is educational, but an SDK is cleaner. Add the official OpenAI Java library as a Maven dependency (from `com.openai`), reload the project, and use the builder pattern. Unlike the raw approach, this SDK code is specific to OpenAI and will not work for other providers.


```
import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.responses.ResponseCreateParams;
import com.openai.models.ChatModel;

// The client reads OPENAI_API_KEY from the environment automatically
OpenAIClient client = OpenAIOkHttpClient.fromEnv();

ResponseCreateParams params = ResponseCreateParams.builder()
    .input("Best food to try in Bangalore")
    .model(ChatModel.GPT_5)
    .build();

var response = client.responses().create(params);
System.out.println(response);
```

Official Java libraries and quickstart: <https://platform.openai.com/docs/libraries>

REAL CLASSROOM LESSON

The SDK demo hit a few errors live: "credentials required but not set" and "invalid API key". The cause was not the code, it was the IntelliJ run configuration. When you run a *different* class, IntelliJ uses a *different* run configuration, so the environment variable set for one class is missing for the other. If you see credential errors, check that the correct run configuration has your key. Method names in fast moving SDKs can also drift from the docs, so trust the IDE's auto complete over a copied snippet.

11 Practical tips and gotchas

- **Token cost is input plus output plus thinking.** You control the input, but the output length is up to the model. "Thinking" (reasoning) models spend extra tokens reasoning, so for simple tasks prefer fast "flash" style models. You can also cap and shape output using structured output, covered in a later session.
- **GitHub Copilot in IntelliJ:** go to Settings, then Plugins, install Copilot, and sign in from your browser. The auto suggestions in the demo came from Copilot plus IntelliJ's built in AI assistant (the free plugin version). Use the Community edition of IntelliJ, everything shown works there.

- **Azure OpenAI** is a partnership between OpenAI and Microsoft Azure that lets companies use OpenAI models (like GPT) through Azure.
- **Debugging a slow AI app:** if responses take minutes, it is almost never the LLM itself. Add timestamps and measure each step (RAG search, external API calls, UI rendering) to find the real bottleneck.
- **Prerequisite for next week:** Spring AI itself does not need separate study. Solid **Spring Boot** fundamentals are enough. If you are rusty, watch a Spring Boot crash course at 1.5x or 2x speed before the next class.

YOU HAVE NOW DONE THE CORE THING

You sent a prompt from your own code to a real AI model and got a response back, both by hand and with an SDK, on both a paid (OpenAI) and a free (Gemini) provider. Everything from here builds on this simple request and response.



Telusko | AI Engineering and Agentic AI with Java and Spring Boot. Notes for Session 02.